INTEGRAL DRAGON

MODULE 3 | SESSION 11

Parallel Orchestration: Agent Teams

Parallel workers help only when the coordination model is sound.

AI-Accelerated Software Development | Foundations Program

Confidential. Prepared exclusively for Ensemble Health Partners.

Session 10 solved isolation. Session 11 solves coordination.

WHAT CHANGED

- Subagents isolate focused delegated work under a lead.
- Agent teams add independent teammates, shared tasks, and peer messaging.
- The gain is real only when multiple tracks can advance in parallel.
- Otherwise the team just adds token cost and orchestration noise.

> The question is not "can we spawn more agents?" The question is "does this work benefit from peers working in parallel?"

Bad fit

Single-file edits, tightly sequential tasks, and work with many dependencies are usually better in one session or with subagents.

Good fit

Cross-layer coordination, competing hypotheses, parallel review lenses, and research work where teammates can advance without blocking each other.

Operator mindset shift

The engineer is no longer steering one worker. The engineer is orchestrating a small system of workers and communication paths.

Agent teams and subagents are different coordination models.

SUBAGENTS

- Hierarchical relationship to the caller
- Results flow back to the main session
- Lower token cost
- Best when only the output matters
- Good for focused reviews, testing, recon, or bounded implementation

CHOOSE BY TASK SHAPE

AGENT TEAMS

- Independent teammates with their own sessions
- Direct peer-to-peer messaging through a mailbox
- Shared task list with assignment and claiming
- Higher token and coordination cost
- Best when discussion, challenge, and parallel movement matter

An agent team is a small coordination system, not just a spawn command.

Lead session

Creates the team, defines the structure, steers the work, reviews plans when required, and remains responsible for shutdown and cleanup.

Teammates

Separate Claude Code instances with their own context windows. They can work independently and message one another directly.

Shared task list

Coordinates pending, in-progress, and completed work. Clean dependencies and ownership boundaries are what keep the team from thrashing.

Mailbox

Enables teammate-to-teammate messaging and direct lead intervention. This is powerful, but it also introduces drift and oversight risk if ignored.

```
Example roles

- UI / UX explorer
- API / architecture owner
- QA / skeptic reviewer

The point is not raw agent count. The point is distinct lenses with non-overlapping work.
```

Use teams only where parallel exploration adds real value.

STRONG USE CASES

- Parallel code review with security, performance, and test lenses
- Debugging with competing hypotheses that challenge one another
- Cross-layer work where frontend, backend, and QA can advance separately
- Research and design exploration with distinct perspectives

WEAK USE CASES

- Same-file edits with high merge collision risk
- Sequential tasks with heavy dependencies
- Tiny fixes where startup overhead dominates
- Unclear work where no one has defined role boundaries yet

A team is justified when independent workstreams can progress without waiting, and when the teammates need to share findings rather than report only to the lead.

Create a small team with explicit roles, then manage it like a real system.

Step 1: Enable and spawn

Enable experimental teams, then ask the lead to create a 3-agent team with named roles and clear ownership boundaries.

Step 2: Inspect the task list

Check whether the lead created enough distinct tasks, whether dependencies are clear, and whether the teammates can self-claim safely.

Step 3: Intervene deliberately

Message a teammate directly, redirect work, require the lead to wait, then shut down and clean up through the lead.

```
Example prompt

Create an agent team for a feature proposal.
Spawn three teammates:
- UX researcher
- API architect
- QA skeptic

Require plan approval for implementation work.
Keep tasks separate by workstream and avoid same-file edits.
```

- The demo should emphasize role separation, not spectacle.
- The lead should coordinate, not silently take over all of the work.
- Cleanup is part of the operational story, not an afterthought.

Parallelism increases leverage, but it also increases cost, drift, and permission surface.

Token cost

Each teammate runs its own context window. Anthropic's current docs explicitly recommend small teams and focused prompts to keep costs manageable.

Coordination drift

More workers means more opportunities for duplicate effort, idle waiting, or partial misunderstandings that compound before the lead notices.

Permissions and quality gates

Teammates inherit the lead's permission settings. Hooks can enforce controls at teammate idle, task creation, and task completion boundaries.

Best-practice controls

Start with research and review, keep teams small, separate file ownership, wait for teammates to finish, and monitor actively rather than walking away.

Agent teams are worth it when parallel coordination is the feature, not just the side effect.

- Use subagents when you need focused workers that report back.
- Use agent teams when teammates need to share findings, challenge each other, and coordinate in parallel.
- Keep teams small, roles explicit, and task boundaries non-overlapping.
- The lead stays accountable for steering, approvals, shutdown, and cleanup.

Parallel orchestration is a control problem as much as it is a capability problem.

BRIDGE TO SESSION 12

Expanded attack surface

Session 11 intentionally expands the execution surface: more sessions, more messaging, more tools, more coordination paths.

Next question

Session 12 pivots to OWASP Top 10 for Agentic Apps and asks what new failure and breach modes appear once agents can act through tools and teams.